In [ ]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.preprocessing import StandardScaler
```

In [ ]:
```
DATA_PATH = "weatherAUS.csv"
```

In [ ]:
```
data = pd.read_csv(DATA_PATH,parse_dates=["Date"])
```

In [ ]:
```
data.head(5)
```

Out[ ]:

	Date	Location	MinTemp	MaxTemp	Rainfall	Evaporation	Sunshine	WindGus
0	2008-12-01	Albury	13.4	22.9	0.6	NaN	NaN	
1	2008-12-02	Albury	7.4	25.1	0.0	NaN	NaN	W
2	2008-12-03	Albury	12.9	25.7	0.0	NaN	NaN	W
3	2008-12-04	Albury	9.2	28.0	0.0	NaN	NaN	
4	2008-12-05	Albury	17.5	32.3	1.0	NaN	NaN	

5 rows × 23 columns

In [ ]:
```
data.dtypes
```

Out[ ]:
```
Date             datetime64[us]
Location                    str
MinTemp                 float64
MaxTemp                 float64
Rainfall                float64
Evaporation             float64
Sunshine                float64
WindGustDir                 str
WindGustSpeed           float64
WindDir9am                  str
WindDir3pm                  str
WindSpeed9am            float64
WindSpeed3pm            float64
Humidity9am             float64
Humidity3pm             float64
Pressure9am             float64
Pressure3pm             float64
Cloud9am                float64
Cloud3pm                float64
Temp9am                 float64
Temp3pm                 float64
RainToday                   str
RainTomorrow                str
dtype: object
```

In [ ]:
```
data["Location"].value_counts()
```

Out[ ]:
```
Location
Canberra            3436
Sydney              3344
Melbourne           3193
Brisbane            3193
Adelaide            3193
Perth               3193
Hobart              3193
Darwin              3193
Albury              3040
Wollongong          3040
MountGinini         3040
Ballarat            3040
Bendigo             3040
Cairns              3040
GoldCoast           3040
Townsville          3040
MountGambier        3040
Albany              3040
Launceston          3040
AliceSprings        3040
Newcastle           3039
Penrith             3039
Tuggeranong         3039
BadgerysCreek       3009
Cobar               3009
CoffsHarbour        3009
Moree               3009
NorfolkIsland       3009
Richmond            3009
SydneyAirport       3009
WaggaWagga          3009
Williamtown         3009
Sale                3009
MelbourneAirport    3009
Mildura             3009
Portland            3009
Watsonia            3009
Dartmoor            3009
Nuriootpa           3009
Woomera             3009
Witchcliffe         3009
PearceRAAF          3009
PerthAirport        3009
Walpole             3006
NorahHead           3004
SalmonGums          3001
Nhil                1578
Katherine           1578
Uluru               1578
Name: count, dtype: int64
```

In [ ]:
```
data_melb_airport = data.loc[data["Location"]=="MelbourneAirport",["Date"
```

In [ ]:
```
data_melb_airport = (
    data_melb_airport
    .sort_values("Date")
```

```
    .reset_index(drop=True)
)
```

In [ ]:
```
data_melb_airport.head(5)
```

Out[ ]:

	Date	MaxTemp
0	2009-01-01	19.9
1	2009-01-02	17.8
2	2009-01-03	21.1
3	2009-01-04	29.2
4	2009-01-05	29.0

In [ ]:
```
data_melb_airport.shape
```

Out[ ]:
```
(3009, 2)
```

In [ ]:
```
data_melb_airport.describe()
```

Out[ ]:

	Date	MaxTemp
count	3009	3009.000000
mean	2013-04-07 02:02:30.747756	20.491858
min	2009-01-01 00:00:00	8.400000
25%	2011-01-23 00:00:00	15.300000
50%	2013-05-13 00:00:00	19.100000
75%	2015-06-04 00:00:00	24.500000
max	2017-06-25 00:00:00	46.800000
std	NaN	6.653313

In [ ]:
```
data_melb_airport.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3009 entries, 0 to 3008
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype         
---  ------   --------------  -----         
 0   Date     3009 non-null   datetime64[us]
 1   MaxTemp  3009 non-null   float64       
dtypes: datetime64[us](1), float64(1)
memory usage: 47.1 KB
```

In [ ]:
```
data_melb_airport.isna().sum()
```

Out[ ]:
```
Date       0
MaxTemp    0
dtype: int64
```

In [ ]:
```
print(
    "Duplicate dates:",
```

```
    data_melb_airport["Date"].duplicated().sum()
)

expected_dates = pd.date_range(
    start=data_melb_airport["Date"].min(),
    end=data_melb_airport["Date"].max(),
    freq="D"
)

missing_dates = expected_dates.difference(
    data_melb_airport["Date"]
)

print("Missing calendar dates:", len(missing_dates))
print(missing_dates)
```

```
Duplicate dates: 0
Missing calendar dates: 89
DatetimeIndex(['2011-04-01', '2011-04-02', '2011-04-03', '2011-04-04',
               '2011-04-05', '2011-04-06', '2011-04-07', '2011-04-08',
               '2011-04-09', '2011-04-10', '2011-04-11', '2011-04-12',
               '2011-04-13', '2011-04-14', '2011-04-15', '2011-04-16',
               '2011-04-17', '2011-04-18', '2011-04-19', '2011-04-20',
               '2011-04-21', '2011-04-22', '2011-04-23', '2011-04-24',
               '2011-04-25', '2011-04-26', '2011-04-27', '2011-04-28',
               '2011-04-29', '2011-04-30', '2012-12-01', '2012-12-02',
               '2012-12-03', '2012-12-04', '2012-12-05', '2012-12-06',
               '2012-12-07', '2012-12-08', '2012-12-09', '2012-12-10',
               '2012-12-11', '2012-12-12', '2012-12-13', '2012-12-14',
               '2012-12-15', '2012-12-16', '2012-12-17', '2012-12-18',
               '2012-12-19', '2012-12-20', '2012-12-21', '2012-12-22',
               '2012-12-23', '2012-12-24', '2012-12-25', '2012-12-26',
               '2012-12-27', '2012-12-28', '2012-12-29', '2012-12-30',
               '2012-12-31', '2013-02-01', '2013-02-02', '2013-02-03',
               '2013-02-04', '2013-02-05', '2013-02-06', '2013-02-07',
               '2013-02-08', '2013-02-09', '2013-02-10', '2013-02-11',
               '2013-02-12', '2013-02-13', '2013-02-14', '2013-02-15',
               '2013-02-16', '2013-02-17', '2013-02-18', '2013-02-19',
               '2013-02-20', '2013-02-21', '2013-02-22', '2013-02-23',
               '2013-02-24', '2013-02-25', '2013-02-26', '2013-02-27',
               '2013-02-28'],
              dtype='datetime64[us]', freq=None)
```

In [ ]:
```
data_melb_airport = (
    data_melb_airport
    .set_index("Date")
    .asfreq("D")
)
```

In [ ]:
```
data_melb_airport.isna().sum()
```

Out[ ]:
```
MaxTemp    89
dtype: int64
```

In [ ]:
```
data_melb_airport.loc["2011-03-28":"2011-05-03"]
```

Out[ ]:

Date	MaxTemp
2011-03-28	24.5
2011-03-29	28.0
2011-03-30	22.0
2011-03-31	17.2
2011-04-01	NaN
2011-04-02	NaN
2011-04-03	NaN
2011-04-04	NaN
2011-04-05	NaN
2011-04-06	NaN
2011-04-07	NaN
2011-04-08	NaN
2011-04-09	NaN
2011-04-10	NaN
2011-04-11	NaN
2011-04-12	NaN
2011-04-13	NaN
2011-04-14	NaN
2011-04-15	NaN
2011-04-16	NaN
2011-04-17	NaN
2011-04-18	NaN
2011-04-19	NaN
2011-04-20	NaN
2011-04-21	NaN
2011-04-22	NaN
2011-04-23	NaN
2011-04-24	NaN
2011-04-25	NaN
2011-04-26	NaN
2011-04-27	NaN
2011-04-28	NaN
2011-04-29	NaN

Date	MaxTemp
2011-04-30	NaN
2011-05-01	17.9
2011-05-02	17.2
2011-05-03	17.1

In [ ]:
```
train_validation,test_data = train_test_split(
    data_melb_airport,
    test_size= 0.15,
    shuffle=False
)

train_data,validation_data = train_test_split(
    train_validation,
    test_size=0.15/0.85,
    shuffle=False
)
```

In [ ]:
```
print("Training:", train_data.index.min(), "to", train_data.index.max())
print("Validation:", validation_data.index.min(), "to", validation_data.i
print("Testing:", test_data.index.min(), "to", test_data.index.max())

print("\nNumber of observations:")
print("Total:",len(data_melb_airport))
print("Training:", len(train_data))
print("Validation:", len(validation_data))
print("Testing:", len(test_data))
```

```
Training: 2009-01-01 00:00:00 to 2014-12-08 00:00:00
Validation: 2014-12-09 00:00:00 to 2016-03-17 00:00:00
Testing: 2016-03-18 00:00:00 to 2017-06-25 00:00:00

Number of observations:
Total: 3098
Training: 2168
Validation: 465
Testing: 465
```

In [ ]:
```
data_melb_airport.head(5)
```

Out[ ]:

Date	MaxTemp
2009-01-01	19.9
2009-01-02	17.8
2009-01-03	21.1
2009-01-04	29.2
2009-01-05	29.0

In [ ]:
```
validation_baseline = validation_data.copy()

validation_baseline["Naive baseline"] = (
    data_melb_airport["MaxTemp"]
    .shift(1)
    .loc[validation_data.index]
)
```

In [ ]:
```
validation_evaluation = validation_baseline.dropna(
    subset=["MaxTemp", "Naive baseline"]
)
```

In [ ]:
```
naive_mae = mean_absolute_error(
    validation_evaluation["MaxTemp"],
    validation_evaluation["Naive baseline"]
)

naive_rmse = mean_squared_error(
    validation_evaluation["MaxTemp"],
    validation_evaluation["Naive baseline"]
) ** 0.5

print(f"Naive validation MAE: {naive_mae:.3f} °C")
print(f"Naive validation RMSE: {naive_rmse:.3f} °C")
```

```
Naive validation MAE: 4.044 °C
Naive validation RMSE: 5.548 °C
```

In [ ]:
```
scaler = StandardScaler()

scaler.fit(
    train_data[["MaxTemp"]].dropna()
)
```

Out[ ]:

▾ StandardScaler ⓘ ?

▸ Parameters

▸ Fitted attributes

▸ 1 feature

In [ ]:
```
scaled_temperature = pd.Series(
    np.nan,
    index=data_melb_airport.index,
    name="ScaledMaxTemp",
    dtype=float
)

available = data_melb_airport["MaxTemp"].notna()

scaled_temperature.loc[available] = (
    scaler.transform(
        data_melb_airport.loc[available, ["MaxTemp"]]
    )
    .ravel()
)
```

In [ ]:
```
scaled_temperature.head(5)
```

Out[ ]:
```
Date
2009-01-01   -0.048412
2009-01-02   -0.370755
2009-01-03    0.135785
2009-01-04    1.379110
2009-01-05    1.348411
Freq: D, Name: ScaledMaxTemp, dtype: float64
```

In [ ]:
```
import copy
import random

import matplotlib.pyplot as plt

import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader


torch.set_num_threads(1)

SEED = 42
```

In [ ]:
```
def make_windows(series, window):
    values = series.to_numpy(dtype=np.float32)

    X = []
    y = []
    target_dates = []

    for i in range(window, len(values)):
        block = values[i - window : i + 1]

        if not np.isfinite(block).all():
            continue

        X.append(block[:-1])
        y.append(block[-1])
        target_dates.append(series.index[i])

    X = np.asarray(X, dtype=np.float32)[..., None]
```

```
    y = np.asarray(y, dtype=np.float32).reshape(-1, 1)

    return X, y, pd.DatetimeIndex(target_dates)
```

In [ ]:
```
def prepare_data(window):
    X, y, dates = make_windows(
        scaled_temperature,
        window=window
    )

    splits = {
        "train": train_data,
        "validation": validation_data,
        "test": test_data
    }

    prepared = {}

    for name, frame in splits.items():
        mask = dates.isin(frame.index)

        prepared[name] = (
            torch.from_numpy(X[mask]),
            torch.from_numpy(y[mask]),
            dates[mask]
        )

    return prepared


example_data = prepare_data(window=7)

for name, (X, y, dates) in example_data.items():
    print(f"{name}: X = {tuple(X.shape)}, y = {tuple(y.shape)}")
```

```
train: X = (2051, 7, 1), y = (2051, 1)
validation: X = (465, 7, 1), y = (465, 1)
test: X = (465, 7, 1), y = (465, 1)
```

In [ ]:
```
class TemperatureGRU(nn.Module):
    def __init__(self, hidden_size=32):
        super().__init__()

        self.gru = nn.GRU(
            input_size=1,
            hidden_size=hidden_size,
            num_layers=1,
            batch_first=True
        )

        self.output = nn.Linear(hidden_size, 1)

    def forward(self, x):
        sequence, _ = self.gru(x)

        final_step = sequence[:, -1, :]

        return self.output(final_step)
```

In [ ]:
```python
def fit_gru(
    data,
    hidden_size=32,
    lr=0.001,
    batch_size=32,
    max_epochs=100,
    patience=10
):
    random.seed(SEED)
    np.random.seed(SEED)
    torch.manual_seed(SEED)

    model = TemperatureGRU(hidden_size=hidden_size)

    optimizer = torch.optim.Adam(
        model.parameters(),
        lr=lr
    )

    criterion = nn.MSELoss()

    X_train, y_train, _ = data["train"]
    X_val, y_val, _ = data["validation"]

    train_loader = DataLoader(
        TensorDataset(X_train, y_train),
        batch_size=batch_size,
        shuffle=True
    )

    best_loss = float("inf")
    best_state = None
    best_epoch = 0
    epochs_without_improvement = 0

    history = {
        "train": [],
        "validation": []
    }

    for epoch in range(max_epochs):
        model.train()
        total_train_loss = 0.0

        for X_batch, y_batch in train_loader:
            optimizer.zero_grad()

            predictions = model(X_batch)
            loss = criterion(predictions, y_batch)

            loss.backward()

            nn.utils.clip_grad_norm_(
                model.parameters(),
                max_norm=1.0
            )

            optimizer.step()
```

```
            total_train_loss += loss.item() * len(X_batch)

        train_loss = total_train_loss / len(X_train)

        model.eval()

        with torch.no_grad():
            val_predictions = model(X_val)
            val_loss = criterion(val_predictions, y_val).item()

        history["train"].append(train_loss)
        history["validation"].append(val_loss)

        if val_loss < best_loss:
            best_loss = val_loss
            best_state = copy.deepcopy(model.state_dict())
            best_epoch = epoch + 1
            epochs_without_improvement = 0
        else:
            epochs_without_improvement += 1

        if epoch == 0 or (epoch + 1) % 10 == 0:
            print(
                f"Epoch {epoch + 1:3d} | "
                f"Train MSE: {train_loss:.4f} | "
                f"Validation MSE: {val_loss:.4f}"
            )

        if epochs_without_improvement >= patience:
            print(f"Early stopping at epoch {epoch + 1}")
            break

    model.load_state_dict(best_state)
    model.eval()

    return model, history, best_epoch
```

In [ ]:

```
def evaluate(model, split):
    X, y, dates = split

    model.eval()

    with torch.no_grad():
        predicted_z = model(X).numpy()

    predicted = scaler.inverse_transform(predicted_z).ravel()
    actual = scaler.inverse_transform(y.numpy()).ravel()

    naive = (
        data_melb_airport["MaxTemp"]
        .shift(1)
        .loc[dates]
        .to_numpy()
    )

    metrics = {
        "GRU MAE": mean_absolute_error(actual, predicted),
        "GRU RMSE": mean_squared_error(actual, predicted) ** 0.5,
        "Naive MAE": mean_absolute_error(actual, naive),
        "Naive RMSE": mean_squared_error(actual, naive) ** 0.5
```

```
    }

    forecasts = pd.DataFrame(
        {
            "Actual": actual,
            "GRU": predicted,
            "Naive": naive
        },
        index=dates
    )

    return metrics, forecasts
```

In [ ]:

```
experiments = {}
comparison_rows = []

for window in [7, 14, 30]:
    print(f"\nTraining GRU with a {window}-day input window")

    prepared = prepare_data(window)

    model, history, best_epoch = fit_gru(
        data=prepared,
        hidden_size=32,
        lr=0.001,
        batch_size=32,
        max_epochs=100,
        patience=10
    )

    val_metrics, _ = evaluate(
        model,
        prepared["validation"]
    )

    experiments[window] = {
        "model": model,
        "data": prepared,
        "history": history,
        "best_epoch": best_epoch
    }

    comparison_rows.append({
        "Window": window,
        "Best epoch": best_epoch,
        **val_metrics
    })

comparison = (
    pd.DataFrame(comparison_rows)
    .set_index("Window")
    .sort_values("GRU RMSE")
)

print("\nValidation comparison — errors in °C")
display(comparison.round(3))

best_window = int(comparison.index[0])
best_run = experiments[best_window]
best_model = best_run["model"]
```

```
print(f"\nSelected input window: {best_window} days")
```

```
Training GRU with a 7-day input window
Epoch   1 | Train MSE: 0.6481 | Validation MSE: 0.5980
Epoch  10 | Train MSE: 0.3621 | Validation MSE: 0.5245
Epoch  20 | Train MSE: 0.3578 | Validation MSE: 0.5265
Epoch  30 | Train MSE: 0.3560 | Validation MSE: 0.5236
Early stopping at epoch 34

Training GRU with a 14-day input window
Epoch   1 | Train MSE: 0.6262 | Validation MSE: 0.5708
Epoch  10 | Train MSE: 0.3507 | Validation MSE: 0.5158
Epoch  20 | Train MSE: 0.3463 | Validation MSE: 0.5047
Epoch  30 | Train MSE: 0.3444 | Validation MSE: 0.5270
Early stopping at epoch 30

Training GRU with a 30-day input window
Epoch   1 | Train MSE: 0.6176 | Validation MSE: 0.5694
Epoch  10 | Train MSE: 0.3394 | Validation MSE: 0.5079
Epoch  20 | Train MSE: 0.3308 | Validation MSE: 0.5003
Epoch  30 | Train MSE: 0.3264 | Validation MSE: 0.5096
Early stopping at epoch 30

Validation comparison — errors in °C
```

Window	Best epoch	GRU MAE	GRU RMSE	Naive MAE	Naive RMSE
30	20	3.508	4.608	4.044	5.548
14	20	3.516	4.628	4.044	5.548
7	24	3.581	4.700	4.044	5.548

```
Selected input window: 30 days
```

In [ ]:
```
results = {}
forecasts = {}

for split_name in ["train", "validation", "test"]:
    metrics, predictions = evaluate(
        best_model,
        best_run["data"][split_name]
    )

    results[split_name] = metrics
    forecasts[split_name] = predictions

results_table = pd.DataFrame.from_dict(
    results,
    orient="index"
)

print("Errors in °C")
display(results_table.round(3))

test_gru_rmse = results["test"]["GRU RMSE"]
test_naive_rmse = results["test"]["Naive RMSE"]

improvement = (
```

```
    (test_naive_rmse - test_gru_rmse)
    / test_naive_rmse
    * 100
)

print(
    f"Test RMSE improvement over naive: {improvement:.2f}% "
)
```

Errors in °C

	GRU MAE	GRU RMSE	Naive MAE	Naive RMSE
train	2.802	3.752	3.086	4.366
validation	3.508	4.608	4.044	5.548
test	2.768	3.673	3.134	4.359

Test RMSE improvement over naive: 15.74%

In [ ]:
```
history = best_run["history"]

epochs = range(1, len(history["train"]) + 1)

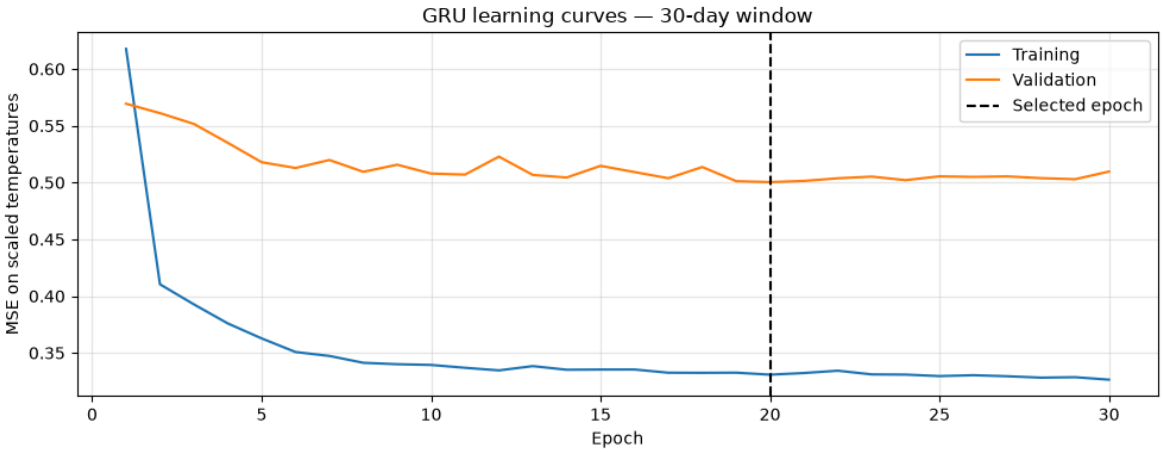
plt.figure(figsize=(10, 4))

plt.plot(
    epochs,
    history["train"],
    label="Training"
)

plt.plot(
    epochs,
    history["validation"],
    label="Validation"
)

plt.axvline(
    best_run["best_epoch"],
    color="black",
    linestyle="--",
    label="Selected epoch"
)

plt.title(f"GRU learning curves — {best_window}-day window")
plt.xlabel("Epoch")
plt.ylabel("MSE on scaled temperatures")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



In [ ]:
```
test_forecasts = forecasts["test"]

fig, axes = plt.subplots(2, 1, figsize=(14, 9))

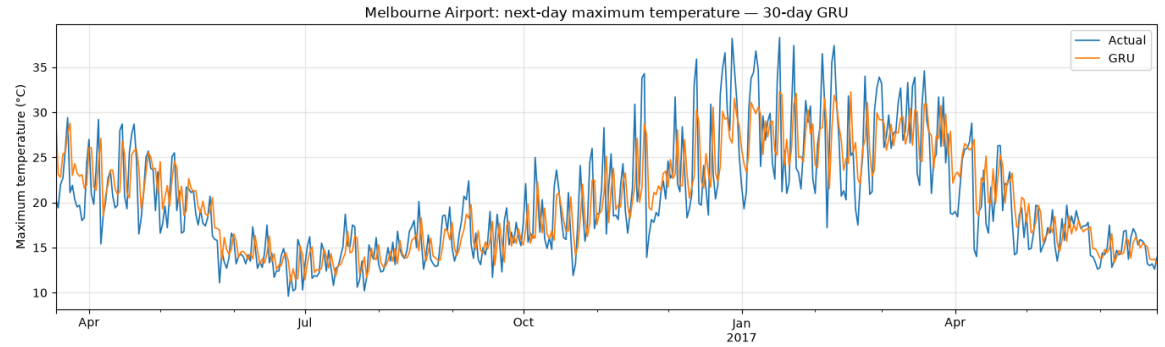
test_forecasts[["Actual", "GRU"]].plot(
    ax=axes[0],
    linewidth=1.2
)

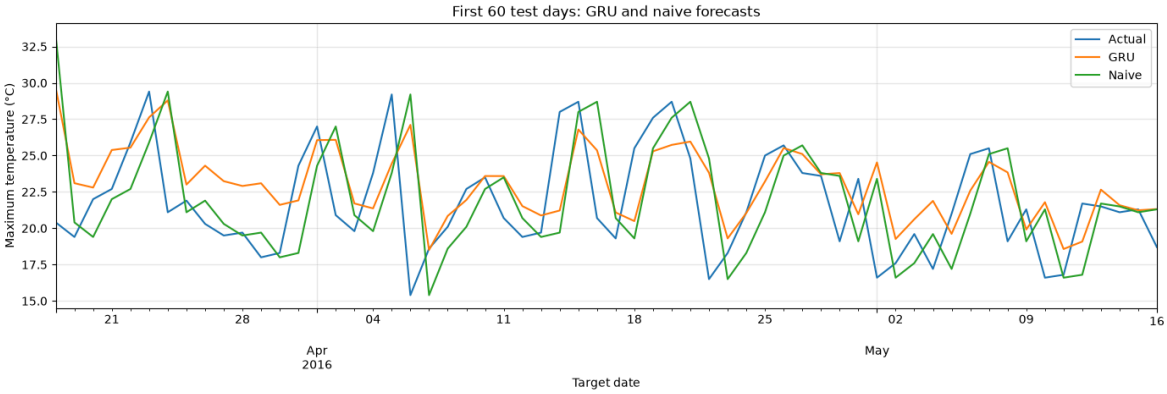
axes[0].set_title(
    f"Melbourne Airport: next-day maximum temperature "
    f"— {best_window}-day GRU"
)
axes[0].set_ylabel("Maximum temperature (°C)")
axes[0].grid(alpha=0.3)

test_forecasts.iloc[:60].plot(
    ax=axes[1],
    linewidth=1.5
)

axes[1].set_title("First 60 test days: GRU and naive forecasts")
axes[1].set_xlabel("Target date")
axes[1].set_ylabel("Maximum temperature (°C)")
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()
```


Melbourne Airport: next-day maximum temperature — 30-day GRU
Maximum temperature (°C)
Actual
GRU
Apr
Jul
Oct
Jan
2017
Apr


First 60 test days: GRU and naive forecasts
Maximum temperature (°C)
Actual
GRU
Naive
Apr
2016
May
Target date